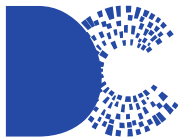




UNIVERSIDADE
FEDERAL DE
SERGIPE



DEPARTAMENTO
DE COMPUTAÇÃO

Plataforma virtual

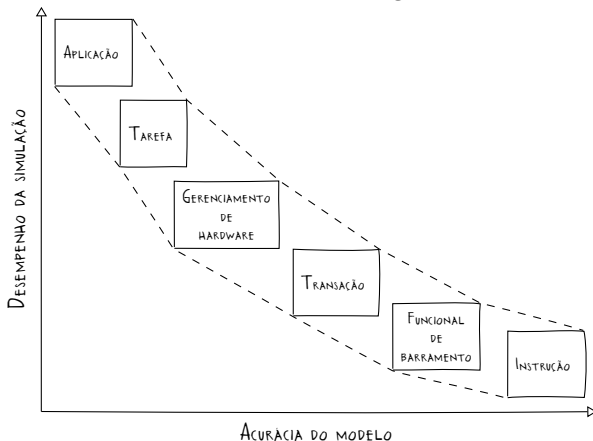
Interface Hardware/Software

Bruno Prado

Departamento de Computação / UFS

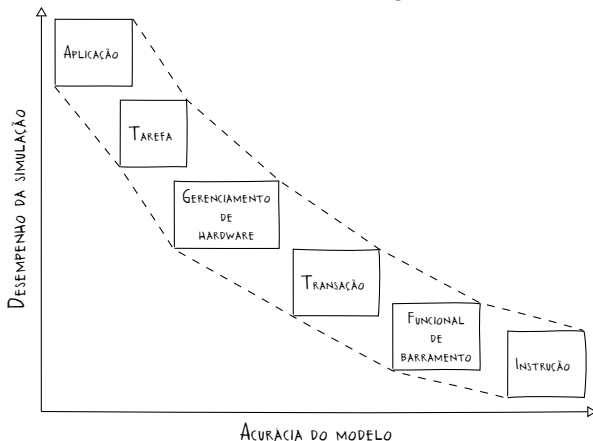
Introdução

- ▶ O que é uma plataforma virtual?
 - ▶ É um ambiente de simulação para prototipação virtual de sistemas (hardware + software integrados)



Introdução

- ▶ O que é uma plataforma virtual?
 - ▶ É um ambiente de simulação para prototipação virtual de sistemas (hardware + software integrados)



"Silício sem software é apenas areia"

Introdução

- ▶ Por que utilizar uma plataforma virtual?
 - ▶ Sistemas complexos de hardware/software
 - ▶ Abstração de implementação
 - ▶ Estimativas de custo e de desempenho

Introdução

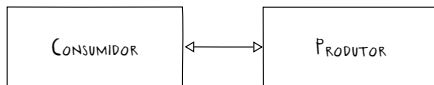
- ▶ Por que utilizar uma plataforma virtual?
 - ▶ Sistemas complexos de hardware/software
 - ▶ Abstração de implementação
 - ▶ Estimativas de custo e de desempenho
 - ▶ Exploração do espaço de projeto
 - ▶ Busca pela solução mais eficiente
 - ▶ Embasar as decisões de projeto

Introdução

- ▶ Por que utilizar uma plataforma virtual?
 - ▶ Sistemas complexos de hardware/software
 - ▶ Abstração de implementação
 - ▶ Estimativas de custo e de desempenho
 - ▶ Exploração do espaço de projeto
 - ▶ Busca pela solução mais eficiente
 - ▶ Embasar as decisões de projeto
 - ▶ Atender objetivos conflitantes
 - ▶ Área $\times$ Custo $\times$ Desempenho $\times$ Potência
 - ▶ Tempo de desenvolvimento $\times$ *Time-to-market* (TTM)

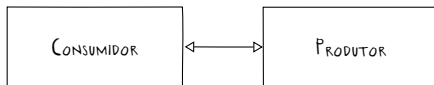
Introdução

- ▶ Fluxo de projeto com plataforma virtual
 - ▶ Nível de aplicação
 - ▶ Descrição do comportamento e comunicação
 - ▶ Independente de plataforma



Introdução

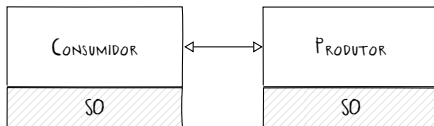
- ▶ Fluxo de projeto com plataforma virtual
 - ▶ Nível de aplicação
 - ▶ Descrição do comportamento e comunicação
 - ▶ Independente de plataforma



Criar uma especificação executável
com alto nível de abstração

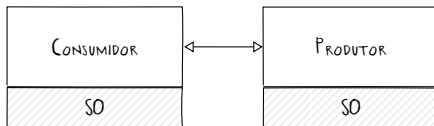
Introdução

- ▶ Fluxo de projeto com plataforma virtual
 - ▶ Nível de tarefas
 - ▶ Definição de interfaces de software
 - ▶ Modelo de SO para a execução das tarefas



Introdução

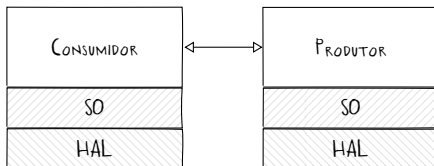
- ▶ Fluxo de projeto com plataforma virtual
 - ▶ Nível de tarefas
 - ▶ Definição de interfaces de software
 - ▶ Modelo de SO para a execução das tarefas



Avaliação de políticas de escalonamento
e mecanismos de sincronização

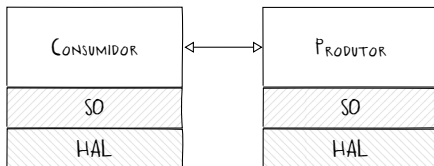
Introdução

- ▶ Fluxo de projeto com plataforma virtual
 - ▶ Nível de gerenciamento de hardware
 - ▶ Acesso ao hardware da plataforma
 - ▶ Camada de abstração de hardware (HAL)



Introdução

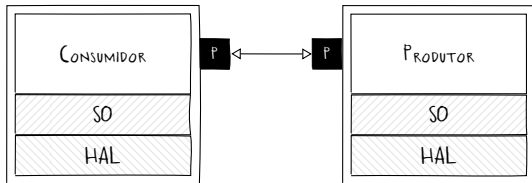
- ▶ Fluxo de projeto com plataforma virtual
 - ▶ Nível de gerenciamento de hardware
 - ▶ Acesso ao hardware da plataforma
 - ▶ Camada de abstração de hardware (HAL)



Análise de parâmetros de configuração dos dispositivos,
como taxa de transmissão ou modo de operação

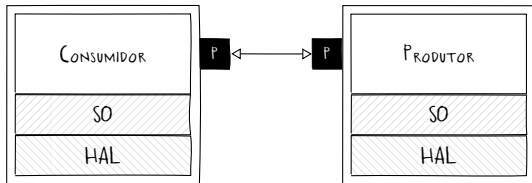
Introdução

- ▶ Fluxo de projeto com plataforma virtual
 - ▶ Nível de transação
 - ▶ Modelagem da comunicação (TLM)
 - ▶ Avaliação de tipos de transmissão e protocolos



Introdução

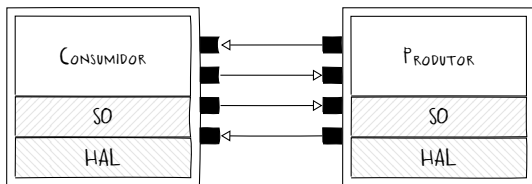
- ▶ Fluxo de projeto com plataforma virtual
 - ▶ Nível de transação
 - ▶ Modelagem da comunicação (TLM)
 - ▶ Avaliação de tipos de transmissão e protocolos



Integração de comportamento e comunicação
em uma especificação executável

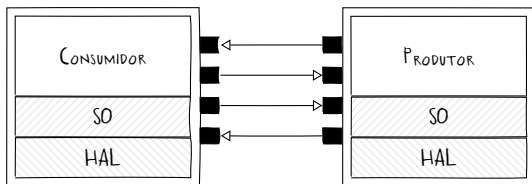
Introdução

- ▶ Fluxo de projeto com plataforma virtual
 - ▶ Nível de modelo funcional de barramento (BFM)
 - ▶ Detalhamento do protocolo de comunicação
 - ▶ Comportamento idêntico ao hardware



Introdução

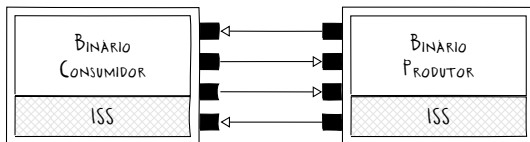
- ▶ Fluxo de projeto com plataforma virtual
 - ▶ Nível de modelo funcional de barramento (BFM)
 - ▶ Detalhamento do protocolo de comunicação
 - ▶ Comportamento idêntico ao hardware



Análise detalhada da comunicação

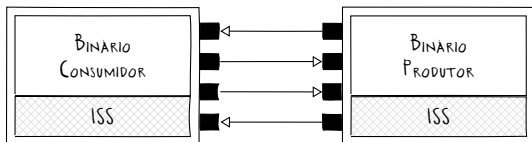
Introdução

- ▶ Fluxo de projeto com plataforma virtual
 - ▶ Nível de instrução (ISS)
 - ▶ Definição da arquitetura de processamento
 - ▶ Precisão funcional ou de ciclo



Introdução

- ▶ Fluxo de projeto com plataforma virtual
 - ▶ Nível de instrução (ISS)
 - ▶ Definição da arquitetura de processamento
 - ▶ Precisão funcional ou de ciclo



Avaliação do comportamento e do desempenho
em uma determinada arquitetura de processador

Introdução

- ▶ Ferramentas de código aberto
 - ▶ ArchC
 - ▶ Gem5
 - ▶ QEMU
 - ▶ Unicorn
 - ▶ *Open Virtual Platforms (OVPsim)**
 - ▶ ...

Prototipação virtual

- ▶ Netduino2 @ QEMU
 - ▶ ARM Cortex-M3
 - ▶ 192 KB de FLASH
 - ▶ 60 KB de RAM
 - ▶ Periféricos
 - ▶ ADC
 - ▶ I2C
 - ▶ SPI
 - ▶ USART
 - ▶ ...

Prototipação virtual

- ▶ Aplicação hello_world
- ▶ Implementação sem SO (*bare metal*)

```
1 // Tipos inteiros padronizados
2 #include <stdint.h>
3 // Registrador da USART
4 volatile uint32_t* const USART =
    (uint32_t*)(0x40011004);
5 // Função de impressão serial
6 void printf_serial(const char* s) {
7     // Imprime até o final da string
8     while(*s != '\\0')
9         *USART = (uint32_t)(*s++);
10 }
11 // Função principal
12 int main() {
13     // Impressão de mensagem no terminal
14     printf_serial("Hello_world_from_QEMU!\\n");
15     // Retornando sucesso
16     return 0;
17 }
```

Prototipação virtual

- ▶ Inicialização do sistema (*bootloader*)
 - ▶ Configuração do hardware e software

```
1 // Definições da arquitetura
2 .syntax unified
3 .cpu cortex-m3
4 .fpu softvfp
5 .thumb
6 // Segmento de código
7 .section .text.bootloader
8 // Tabela de vetor de interrupção
9 .global _vector
10 _vector:
11     .word _estack
12     .word _start
13     .align 9
14 // Função start
15 .thumb_func
16 _start:
17     bl main
18     b .
```

Prototipação virtual

- Configuração dos segmentos na memória
 - Script de linkagem

```
1  /* Endereços de memória e tamanhos */
2  RAM_origin   = 0x20000000;
3  RAM_length   = 60K;
4  FLASH_origin = 0x08000000;
5  FLASH_length = 192K;
6  _estack      = RAM_origin + RAM_length;
7  /* Mapeamento de memória */
8  MEMORY {
9      RAM      : ORIGIN = RAM_origin,   LENGTH = RAM_length
10     FLASH    : ORIGIN = FLASH_origin,  LENGTH = FLASH_length
11 }
12 /* Alocação dos segmentos */
13 SECTIONS {
14     .text     : { *(.text.bootloader) *(.text*) } > FLASH
15     .rodata   : { *(.rodata*) } > FLASH
16     .data     : { *(.data*) } > RAM AT > FLASH
17     .bss      : { *(.bss*) } > RAM
18 }
```

Prototipação virtual

► Regras de construção e execução (*Makefile*)

```
1  # Diretivas de compilação
2  ARM_PREFIX      := arm-none-eabi
3  ARM_AS          := $(ARM_PREFIX)-as
4  ARM_CC          := $(ARM_PREFIX)-gcc
5  ARM_LD          := $(ARM_PREFIX)-ld
6  ARM_OC          := $(ARM_PREFIX)-objcopy
7  ARM_AS_OPTIONS  := -mcpu=cortex-m3 -mthumb -g
8  ARM_CC_OPTIONS  := -Wall -O3 -nostdlib -nostartfiles
   -mcpu=cortex-m3 -mthumb -c -g
9  # Regras de execução
10 all: vp1.bin
11 run: vp1.bin
12     qemu-system-arm -M netduino2 -nographic -kernel $(<)
13 clean:
14     @rm -rf *.o *.elf *.bin
15 start.o: start.s
16     $(ARM_AS) $(ARM_AS_OPTIONS) $(<) -o $(@)
17 vp1.o: vp1.c
18     $(ARM_CC) $(ARM_CC_OPTIONS) $(<) -o $(@)
19 vp1.elf: vp1.ld start.o vp1.o
20     $(ARM_LD) -T $(^) -o $(@)
21 vp1.bin: vp1.elf
22     $(ARM_OC) -O binary $(<) $(@)
```


Prototipação virtual

► Simulação da aplicação

```
$ make
arm-eabi-as -mcpu=cortex-m3 -mthumb -g start.s -o start.o
arm-eabi-gcc -Wall -O3 -nostdlib -nostartfiles -mcpu=cortex-m3 -mthumb -c -g vp1.c -o
vp1.o
arm-eabi-ld -T vp1.ld start.o vp1.o -o vp1.elf
arm-eabi-objcopy -O binary vp1.elf vp1.bin
```

Prototipação virtual

► Simulação da aplicação

```
$ make
arm-eabi-as -mcpu=cortex-m3 -mthumb -g start.s -o start.o
arm-eabi-gcc -Wall -O3 -nostdlib -nostartfiles -mcpu=cortex-m3 -mthumb -c -g vp1.c -o
vp1.o
arm-eabi-ld -T vp1.ld start.o vp1.o -o vp1.elf
arm-eabi-objcopy -O binary vp1.elf vp1.bin
$ make run
qemu-system-arm -M netduino2 -nographic -kernel vp1.bin
Hello world from QEMU!
```

Prototipação virtual

► Simulação da aplicação

```
$ make
arm-eabi-as -mcpu=cortex-m3 -mthumb -g start.s -o start.o
arm-eabi-gcc -Wall -O3 -nostdlib -nostartfiles -mcpu=cortex-m3 -mthumb -c -g vp1.c -o
vp1.o
arm-eabi-ld -T vp1.ld start.o vp1.o -o vp1.elf
arm-eabi-objcopy -O binary vp1.elf vp1.bin
$ make run
qemu-system-arm -M netduino2 -nographic -kernel vp1.bin
Hello world from QEMU!
[Control-a x]
```

Prototipação virtual

► Simulação da aplicação

```
$ make
arm-eabi-as -mcpu=cortex-m3 -mthumb -g start.s -o start.o
arm-eabi-gcc -Wall -O3 -nostdlib -nostartfiles -mcpu=cortex-m3 -mthumb -c -g vp1.c -o
vp1.o
arm-eabi-ld -T vp1.ld start.o vp1.o -o vp1.elf
arm-eabi-objcopy -O binary vp1.elf vp1.bin
$ make run
qemu-system-arm -M netduino2 -nographic -kernel vp1.bin
Hello world from QEMU!
[Control-a x]
QEMU: Terminated
$
```

Prototipação virtual

► Simulação da aplicação

```
$ make
arm-eabi-as -mcpu=cortex-m3 -mthumb -g start.s -o start.o
arm-eabi-gcc -Wall -O3 -nostdlib -nostartfiles -mcpu=cortex-m3 -mthumb -c -g vp1.c -o
vp1.o
arm-eabi-ld -T vp1.ld start.o vp1.o -o vp1.elf
arm-eabi-objcopy -O binary vp1.elf vp1.bin
$ make run
qemu-system-arm -M netduino2 -nographic -kernel vp1.bin
Hello world from QEMU!
[Control-a x]
QEMU: Terminated
$ make clean
$
```

Prototipação virtual

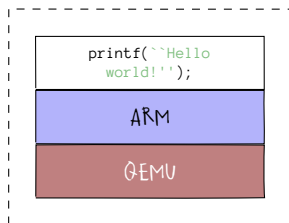
► Simulação da aplicação

```
$ make
arm-eabi-as -mcpu=cortex-m3 -mthumb -g start.s -o start.o
arm-eabi-gcc -Wall -O3 -nostdlib -nostartfiles -mcpu=cortex-m3 -mthumb -c -g vp1.c -o
vp1.o
arm-eabi-ld -T vp1.ld start.o vp1.o -o vp1.elf
arm-eabi-objcopy -O binary vp1.elf vp1.bin
$ make run
qemu-system-arm -M netduino2 -nographic -kernel vp1.bin
Hello world from QEMU!
[Control-a x]
QEMU: Terminated
$ make clean
$ exit
```

Interface de emulação

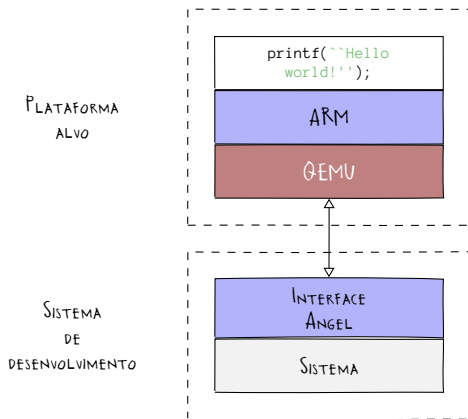
- ▶ Permite ao ambiente de simulação acessar os recursos do sistema de desenvolvimento como se fossem da plataforma alvo (*semihosting*)

PLATAFORMA
ALVO



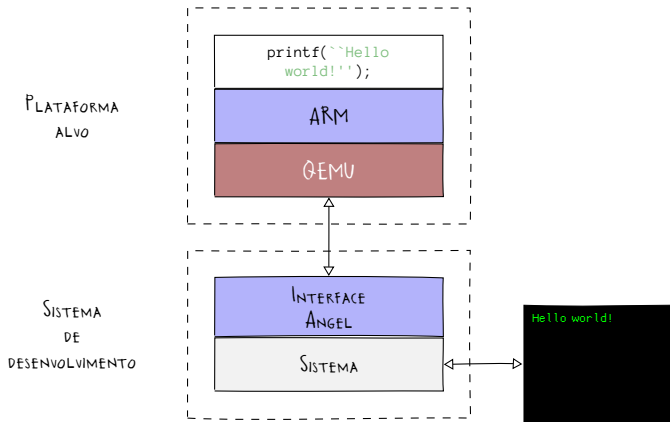
Interface de emulação

- ▶ Permite ao ambiente de simulação acessar os recursos do sistema de desenvolvimento como se fossem da plataforma alvo (*semihosting*)



Interface de emulação

- ▶ Permite ao ambiente de simulação acessar os recursos do sistema de desenvolvimento como se fossem da plataforma alvo (*semihosting*)



Interface de emulação

- ▶ Aplicação hello_world
 - ▶ Implementação sem SO (*bare metal*)

```
1 // E/S padrão
2 #include <stdio.h>
3 // Função principal
4 int main() {
5     // Interação com host
6     char name[50] = "Nobody";
7     FILE* file = fopen("name", "r");
8     printf("What_is_your_name?_");
9     fprintf(stdout, "Reading_from_file...\n");
10    fscanf(file, "%s", name);
11    printf("My_name_is_%s\n", name);
12    fprintf(stderr, "I_am_alive!!!\n");
13    fclose(file);
14    // Retornando sucesso
15    return 0;
16 }
```

Interface de emulação

- ▶ Inicialização do sistema (*bootloader*)
 - ▶ Configuração do hardware e software

```
1 // Definições da arquitetura
2 .syntax unified
3 .cpu cortex-m3
4 .fpu softvfp
5 .thumb
6 // Segmento de código
7 .section .text.bootloader
8 // Tabela de vetor de interrupção
9 .global _vector
10 _vector:
11     .word _estack
12 ...
41 // Função start
42 .thumb_func
43 _start:
44     bl _data_init
45     bl _init_stdio
46     bl main
47     bl _exit
```

Interface de emulação

- Implementação das chamadas de sistema (syscalls)
- Interface ARM Angel

```
1 // Códigos de erro
2 #include <errno.h>
...
8 #define R0(x)    asm volatile("mov_r0,_%[input]" : :
    [input] "i"(x) : "r0")
9 #define R1(x)    asm volatile("mov_r1,_%[input]" : :
    [input] "r"(x) : "r1")
10 #define BKPT()  asm("bkpt_0xAB")
...
102 // Syscall write
103 int _write(int file, char* ptr, int len) {
104     register int value asm("r0");
105     uint32_t p[] = { file, (uint32_t)(ptr), len };
106     R0(0x05);
107     R1(p);
108     BKPT();
109     return value;
110 }
```

Interface de emulação

- Configuração dos segmentos na memória
- Script de linkagem

```
1  /* Ponto de entrada */
2  ENTRY(_start)
3  /* Endereços de memória e tamanhos */
4  RAM_origin   = 0x20000000;
5  RAM_length   = 60K;
6  FLASH_origin = 0x08000000;
7  FLASH_length = 192K;
8  _estack      = RAM_origin + RAM_length;
9  /* Mapeamento de memória */
10 MEMORY {
11     RAM      : ORIGIN = RAM_origin,   LENGTH = RAM_length
12     FLASH    : ORIGIN = FLASH_origin,  LENGTH = FLASH_length
13 }
14 /* Alocação dos segmentos */
15 SECTIONS {
16     .text : {
17         . = ALIGN(4);
18         *(.text.bootloader)
19         ...
20     } > FLASH
21     ...
22 }
90
```

Interface de emulação

- ▶ Arquivo de especificações
 - ▶ Diretivas do *frontend* do GCC

```
1 *link:  
2 -T vp2.ld  
3  
4 *startfile:  
5 start.o  
6  
7 *lib:  
8 -lc
```

Interface de emulação

► Regras de construção e execução (*Makefile*)

```
1  # Diretivas de compilação
...  ...
10 # Regras de execução
11 all: vp2.bin
12 run: vp2.bin
13     qemu-system-arm -M netduino2 -nographic
        -semihosting -kernel $(<)
14 debug: vp2.bin
15     qemu-system-arm -M netduino2 -nographic
        -semihosting -s -S -kernel $(<)
...  ...
22 vp2.elf: vp2.ld $(S_SOURCES:.s=.o) $(C_SOURCES:.c=.o)
23     $(ARM_CC) $(ARM_CC_OPTIONS) $(filter-out
        start.o,$(filter %.o,$(^))) -o $(@)
24 vp2.bin: vp2.elf
25     $(ARM_OC) -O binary $(<) $(@)
```

Interface de emulação

► Simulação da aplicação

```
$ make
arm-none-eabi-as -mcpu=cortex-m3 -mthumb -g start.s -o start.o
arm-none-eabi-gcc -specs=vp2.specs -Wall -O3 -mcpu=cortex-m3 -mthumb -g -c syscall.c
-o syscall.o
arm-none-eabi-gcc -specs=vp2.specs -Wall -O3 -mcpu=cortex-m3 -mthumb -g -c vp2.c -o
vp2.o
arm-none-eabi-gcc -specs=vp2.specs -Wall -O3 -mcpu=cortex-m3 -mthumb -g syscall.o
vp2.o -o vp2.elf
arm-none-eabi-objcopy -O binary vp2.elf vp2.bin
```


Interface de emulação

► Simulação da aplicação

```
$ make
arm-none-eabi-as -mcpu=cortex-m3 -mthumb -g start.s -o start.o
arm-none-eabi-gcc -specs=vp2.specs -Wall -O3 -mcpu=cortex-m3 -mthumb -g -c syscall.c
-o syscall.o
arm-none-eabi-gcc -specs=vp2.specs -Wall -O3 -mcpu=cortex-m3 -mthumb -g -c vp2.c -o
vp2.o
arm-none-eabi-gcc -specs=vp2.specs -Wall -O3 -mcpu=cortex-m3 -mthumb -g syscall.o
vp2.o -o vp2.elf
arm-none-eabi-objcopy -O binary vp2.elf vp2.bin
$ make run
qemu-system-arm -M netduino2 -nographic -semihosting -kernel vp2.bin
```

Interface de emulação

► Simulação da aplicação

```
$ make
arm-none-eabi-as -mcpu=cortex-m3 -mthumb -g start.s -o start.o
arm-none-eabi-gcc -specs=vp2.specs -Wall -O3 -mcpu=cortex-m3 -mthumb -g -c syscall.c
-o syscall.o
arm-none-eabi-gcc -specs=vp2.specs -Wall -O3 -mcpu=cortex-m3 -mthumb -g -c vp2.c -o
vp2.o
arm-none-eabi-gcc -specs=vp2.specs -Wall -O3 -mcpu=cortex-m3 -mthumb -g syscall.o
vp2.o -o vp2.elf
arm-none-eabi-objcopy -O binary vp2.elf vp2.bin
$ make run
qemu-system-arm -M netduino2 -nographic -semihosting -kernel vp2.bin
What is your name?
Reading from file...
My name is Turing
I am alive!!!
$
```

Interface de emulação

► Simulação da aplicação

```
$ make
arm-none-eabi-as -mcpu=cortex-m3 -mthumb -g start.s -o start.o
arm-none-eabi-gcc -specs=vp2.specs -Wall -O3 -mcpu=cortex-m3 -mthumb -g -c syscall.c
-o syscall.o
arm-none-eabi-gcc -specs=vp2.specs -Wall -O3 -mcpu=cortex-m3 -mthumb -g -c vp2.c -o
vp2.o
arm-none-eabi-gcc -specs=vp2.specs -Wall -O3 -mcpu=cortex-m3 -mthumb -g syscall.o
vp2.o -o vp2.elf
arm-none-eabi-objcopy -O binary vp2.elf vp2.bin
$ make run
qemu-system-arm -M netduino2 -nographic -semihosting -kernel vp2.bin
What is your name?
Reading from file...
My name is Turing
I am alive!!!
$ make clean
$
```

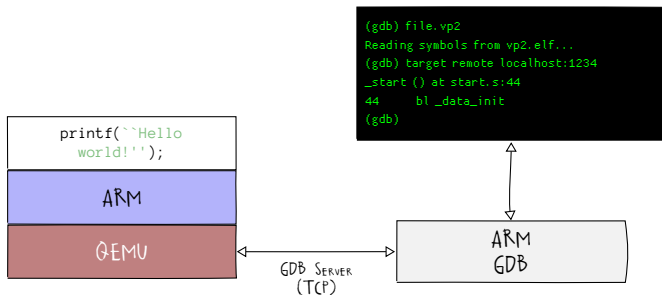
Interface de emulação

► Simulação da aplicação

```
$ make
arm-none-eabi-as -mcpu=cortex-m3 -mthumb -g start.s -o start.o
arm-none-eabi-gcc -specs=vp2.specs -Wall -O3 -mcpu=cortex-m3 -mthumb -g -c syscall.c
-o syscall.o
arm-none-eabi-gcc -specs=vp2.specs -Wall -O3 -mcpu=cortex-m3 -mthumb -g -c vp2.c -o
vp2.o
arm-none-eabi-gcc -specs=vp2.specs -Wall -O3 -mcpu=cortex-m3 -mthumb -g syscall.o
vp2.o -o vp2.elf
arm-none-eabi-objcopy -O binary vp2.elf vp2.bin
$ make run
qemu-system-arm -M netduino2 -nographic -semihosting -kernel vp2.bin
What is your name?
Reading from file...
My name is Turing
I am alive!!!
$ make clean
$ exit
```

Depuração

- ▶ *GNU Debugger* (GDB)
 - ▶ Depuração remota



Depuração

- ▶ Principais opções de depuração do QEMU (**-d**)
 - ▶ *in_asm*: código de montagem da arquitetura alvo
 - ▶ *out_asm*: código de montagem nativo gerado
 - ▶ *int*: interrupções/exceções geradas
 - ▶ *exec*: fluxo de execução da aplicação
 - ▶ *cpu*: valores dos registradores do processador
 - ▶ *fpu*: inclui os registradores da FPU na opção *cpu*
 - ▶ ...

Depuração

► Depuração do QEMU

```
$ qemu-system-arm -d int -M netduino2 -nographic -semihosting -kernel vp2.bin
```

Depuração

► Depuração do QEMU

```
$ qemu-system-arm -d int -M netduino2 -nographic -semihosting -kernel vp2.bin
...
What is your name?
Taking exception 16 [Semihosting call]
...handling as semihosting call 0x5
Reading from file...
Taking exception 16 [Semihosting call]
...handling as semihosting call 0xc
Taking exception 16 [Semihosting call]
...handling as semihosting call 0x6
Taking exception 16 [Semihosting call]
...handling as semihosting call 0xc
Taking exception 16 [Semihosting call]
...handling as semihosting call 0x6
Taking exception 16 [Semihosting call]
...handling as semihosting call 0x5
My name is Turing
Taking exception 16 [Semihosting call]
...handling as semihosting call 0x5
I am alive!!!
...
```


Depuração

- ▶ Parâmetros da aplicação alvo no QEMU
 - ▶ Criar servidor do GDB para conexão remota na porta 1234 (**-s** ou **-gdb tcp::1234**)
 - ▶ Pausa o simulador durante a inicialização (**-S**), aguardando por comandos do GDB

Depuração

- ▶ Parâmetros da aplicação alvo no QEMU
 - ▶ Criar servidor do GDB para conexão remota na porta 1234 (**-s** ou **-gdb tcp::1234**)
 - ▶ Pausa o simulador durante a inicialização (**-S**), aguardando por comandos do GDB
- ▶ Comandos no GDB da plataforma alvo
 - ▶ **file vp2.elf** (informações de depuração)
 - ▶ **target remote localhost:1234** (conexão com QEMU)

Depuração

- QEMU (esquerda) + GDB ARM (direita)

```
$ make debug  
qemu-system-arm -M netduino2 -nographic  
-semihosting -s -S -kernel vp2.bin
```

```
$ arm-eabi-gdb  
...  
For help, type "help".  
Type "apropos word" to search for com-  
mands related to "word".  
(gdb)
```

Depuração

- ▶ QEMU (esquerda) + GDB ARM (direita)

```
$ make debug  
qemu-system-arm -M netduino2 -nographic  
-semihosting -s -S -kernel vp2.bin
```

```
$ arm-eabi-gdb  
...  
For help, type "help".  
Type "apropos word" to search for com-  
mands related to "word".  
(gdb) file vp2.elf  
Reading symbols from vp2.elf...
```

Depuração

- ▶ QEMU (esquerda) + GDB ARM (direita)

```
$ make debug  
qemu-system-arm -M netduino2 -nographic  
-semihosting -s -S -kernel vp2.bin
```

```
$ arm-eabi-gdb  
...  
For help, type "help".  
Type "apropos word" to search for com-  
mands related to "word".  
(gdb) file vp2.elf  
Reading symbols from vp2.elf...  
(gdb) target remote localhost:1234  
Remote debugging using localhost:1234  
_start () at start.s:44  
44 bl _data_init
```

Depuração

- QEMU (esquerda) + GDB ARM (direita)

```
$ make debug
qemu-system-arm -M netduino2 -nographic
-semihosting -s -S -kernel vp2.bin
```

```
$ arm-eabi-gdb
...
For help, type "help".
Type "apropos word" to search for com-
mands related to "word".
(gdb) file vp2.elf
Reading symbols from vp2.elf...
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
_start () at start.s:44
44 bl _data_init
(gdb) c
Continuing.
I am alive!!!
```

Depuração

- QEMU (esquerda) + GDB ARM (direita)

```
$ make debug
qemu-system-arm -M netduino2 -nographic
-semihosting -s -S -kernel vp2.bin
$
```

```
$ arm-eabi-gdb
...
For help, type "help".
Type "apropos word" to search for com-
mands related to "word".
(gdb) file vp2.elf
Reading symbols from vp2.elf...
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
_start () at start.s:44
44 bl _data_init
(gdb) c
Continuing.
I am alive!!!
[Inferior 1 (process 1) exited normally]
(gdb)
```

Depuração

- QEMU (esquerda) + GDB ARM (direita)

```
$ make debug
qemu-system-arm -M netduino2 -nographic
-semihosting -s -S -kernel vp2.bin
$
```

```
$ arm-eabi-gdb
...
For help, type "help".
Type "apropos word" to search for com-
mands related to "word".
(gdb) file vp2.elf
Reading symbols from vp2.elf...
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
_start () at start.s:44
44 bl _data_init
(gdb) c
Continuing.
I am alive!!!
[Inferior 1 (process 1) exited normally]
(gdb) quit
$
```


Exercício

- ▶ Reproduza todos os passos realizados (simulação, emulação e depuração) no QEMU com a plataforma Netduino2
 - ▶ Estude a arquitetura ARM Cortex-M3
 - ▶ Instale/construa o GCC para ARM com suporte para linha R e M de processadores, além do QEMU
 - ▶ Verifique a documentação do QEMU para a plataforma e o código fonte dos componentes
- ▶ Escolha outra plataforma suportada pelo QEMU para realização dos mesmos experimentos
 - ▶ Instale/construa o GCC para arquitetura escolhida
 - ▶ Entenda como funciona a interface de emulação implementada pelo QEMU (*semihosting*)
 - ▶ Realize as modificações necessárias para suportar as especificações da plataforma escolhida